

1. Lab Overview

In this lab, I tested a public REST API using Postman to practice common technical support tasks such as sending GET and POST requests, reading JSON responses, identifying request errors, troubleshooting bad requests, and documenting the resolution in a ticket-style format.

The lab uses **JSONPlaceholder**, a free fake REST API designed for testing and prototyping. JSONPlaceholder supports common API request methods including **GET**, **POST**, **PUT**, **PATCH**, **DELETE**, and **OPTIONS**, making it useful for API support practice.

2. Business Scenario

A customer reports that they are trying to connect their application to a web service API, but their API request is not returning the expected result. They need support confirming whether the endpoint is working, whether the request method is correct, whether the JSON response is readable, and why an incorrect request returns an error.

As a Technical Support Specialist, I need to test the API using Postman, reproduce the customer's issue, troubleshoot the request, document the steps taken, and provide a clear step-by-step resolution.

3. Lab Objectives

By the end of this lab, I will be able to:

- Test a public REST API using Postman.
- Send a successful **GET** request.
- Send a successful **POST** request with a JSON body.
- Read and interpret JSON response data.
- Troubleshoot incorrect endpoints and request errors.
- Document troubleshooting steps in a ticket-style format.
- Explain the resolution clearly for a non-technical customer.

4. Lab Environment

Item	Details
Computer	Windows or macOS
Application	Postman
API Used	JSONPlaceholder

API Type	Public REST API
Authentication	Not required
Internet	Required
Browser	Chrome, Edge, Safari, or Firefox
Documentation Format	GitHub README / Lab Report

5. Tools Used

- **Postman** — to send API requests and review responses.
- **JSONPlaceholder** — public fake REST API for testing.
- **Web Browser** — to review API documentation.
- **GitHub** — to document the lab.
- **Screenshot Tool** — macOS screenshot or Windows Snipping Tool.

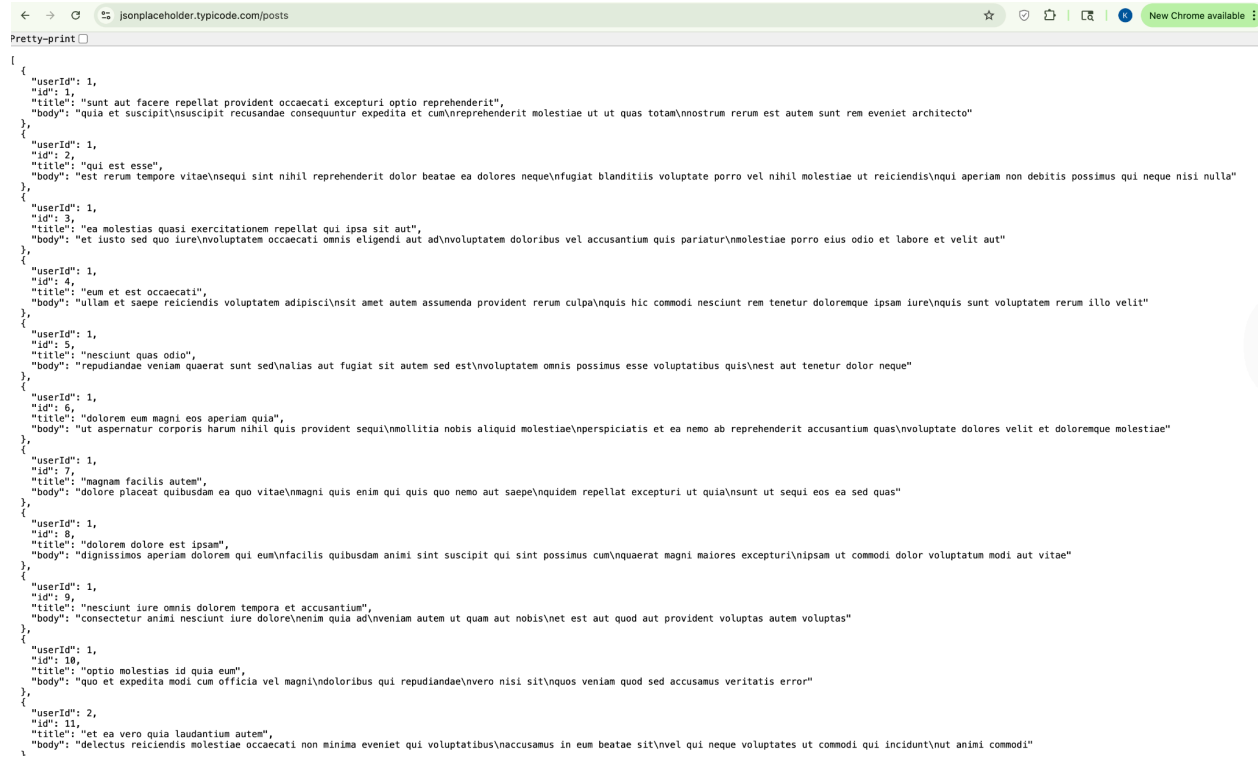
6. Steps Performed with Screenshot Instructions

Step 1: Open JSONPlaceholder API Documentation

Go to the JSONPlaceholder website and review the available API endpoints.

<https://jsonplaceholder.typicode.com/posts>

JSONPlaceholder provides fake API data for testing and prototyping, and its guide shows examples for working with API requests.



Screenshot 1: Public JSONPlaceholder API endpoint returning JSON data in the browser.

Step 2: Open Postman and Create a New Request

Open Postman.

Click:

New > HTTP Request

Select request method:

GET

Enter the request URL:

<https://jsonplaceholder.typicode.com/posts>

Click Send.

Result:

- **Status: 200 OK**
- **Response body: JSON list of posts**

- **Response format: Array of JSON objects**

The screenshot shows a REST client interface with the following details:

- Request:** GET `https://jsonplaceholder.typicode.com/posts`
- Response Status:** 200 OK, 209 ms, 7.98 KB
- Response Body (JSON):**

```
[
  {
    "userId": 1,
    "id": 9,
    "title": "nesciunt iure omnis dolorem tempora et accusantium",
    "body": "consectetur animi nesciunt iure dolore\nenim quia ad\nveniam autem ut quam aut nobis\net est aut quod aut provident voluptas autem voluptas"
  },
  {
    "userId": 1,
    "id": 10,
    "title": "autio molestias id quia eum"
  }
]
```

Screenshot 2: Successful GET request returning JSON data from the /posts endpoint.

Step 3: Read the JSON Response

Review the JSON response.

You should see data similar to this structure:

```
[
{
  "userId": 1,
  "id": 1,
  "title": "sample title",
  "body": "sample body text"
```

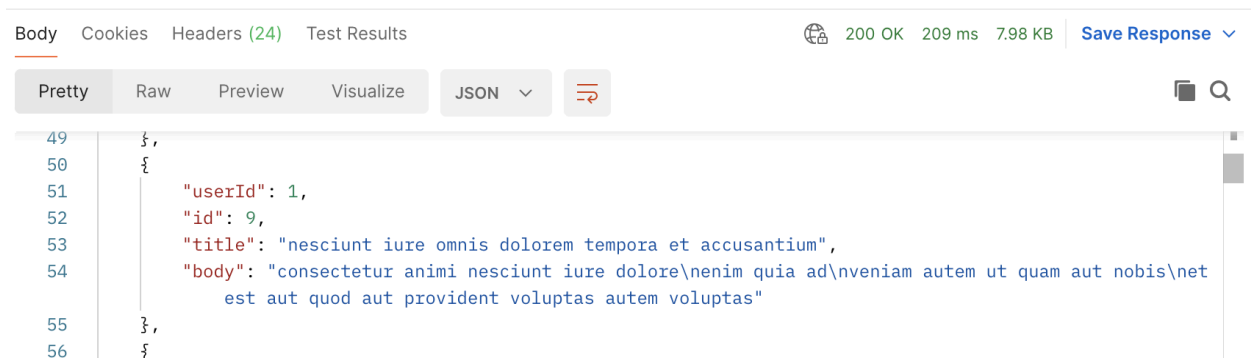
```
}  
]
```

Key fields:

Field	Meaning
<code>userId</code>	User who created the post
<code>id</code>	Unique post ID
<code>title</code>	Post title
<code>body</code>	Post content

Support explanation:

The API returned structured JSON data successfully. This confirms that the endpoint is active and reachable.



Screenshot 3: JSON response showing userId, id, title, and body fields.

Step 4: Send a GET Request for One Specific Post

Create another request in Postman.

Method:

GET

URL:

https://jsonplaceholder.typicode.com/posts/1

Click **Send**.

Result:

- **Status:** **200 OK**
- **Response body:** one JSON object
- **Post ID:** **1**

The screenshot shows the Postman interface for a GET request to `https://jsonplaceholder.typicode.com/posts/1`. The request is configured with the method `GET` and the URL. The response is displayed in the 'Body' tab, showing a JSON object with the following structure:

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae
6     ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
```

Screenshot 4: Successful GET request retrieving one specific post by ID.

Step 4: Send a GET Request for One Specific Post

Create another request in Postman.

Name:

GET Single Post

Method:

GET

URL:

<https://jsonplaceholder.typicode.com/posts/1>

Click **Send**.

Result:

- **Status: 200 OK**
- **Response body: one JSON object**
- **Post ID: 1**

HTTP <https://jsonplaceholder.typicode.com/posts/1> Save

GET <https://jsonplaceholder.typicode.com/posts/1> Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (24) Test Results 200 OK 100 ms 1.26 KB Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas ut"
6 }
```

Screenshot 4: Successful GET request retrieving one specific post by ID.

Step 5: Create a POST request

In Postman, create a new request or change the existing request.

Set method to:

POST

URL:

<https://jsonplaceholder.typicode.com/posts>

Go to:

Body > raw > JSON

Paste this:

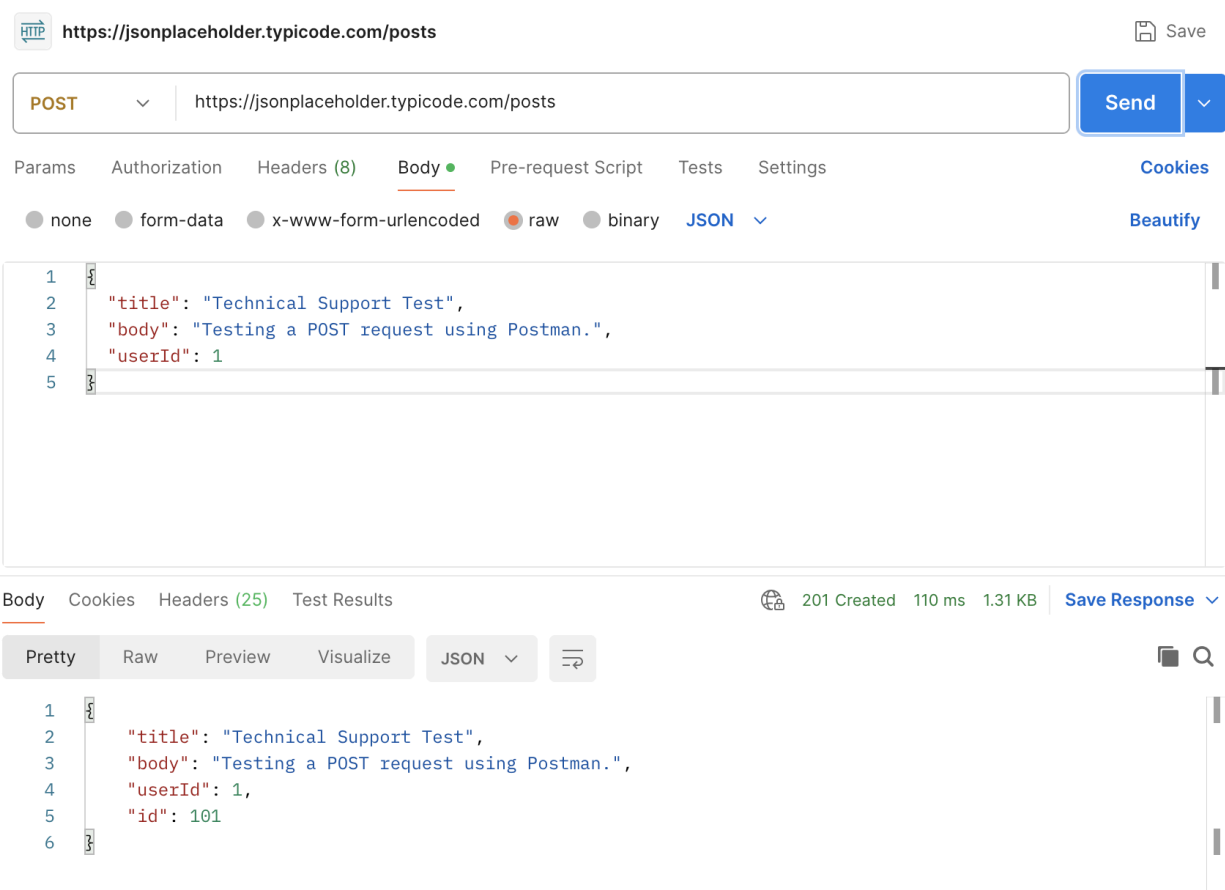
```
{  
  "title": "Technical Support Test",  
  "body": "Testing a POST request using Postman.",  
  "userId": 1  
}
```

Then click **Send**.

Result:

Status: 201 Created

JSONPlaceholder documentation shows that creating a post uses **POST /posts** with a JSON request body, and a successful creation returns a **201** response.



Screenshot 5: Successful POST request creating a test post using JSON body data.

Step 6: Create an error with a bad endpoint

In Postman, set method to:

GET

Wrong URL:

`https://jsonplaceholder.typicode.com/postss`

Click **Send**.

Result:

404 Not Found

Troubleshooting explanation:

The issue occurred because the endpoint path was typed incorrectly as **/postss** instead of **/posts**. After correcting the endpoint, the request returned a successful response.

The screenshot shows a REST client interface with the following details:

- URL:** `https://jsonplaceholder.typicode.com/postss`
- Method:** `GET`
- Buttons:** `Send`, `Save`, `Params`, `Authorization`, `Headers (6)`, `Body`, `Pre-request Script`, `Tests`, `Settings`, `Cookies`
- Query Params Table:**

Key	Value	Bulk Edit
Key	Value	
- Response Section:**
 - Body:** `404 Not Found`, `93 ms`, `1.08 KB`, `Save Response`
 - Buttons:** `Pretty`, `Raw`, `Preview`, `Visualize`, `JSON`, `Copy`, `Search`
 - Line 1:** `{}`

Screenshot 6: Failed GET request caused by incorrect API endpoint path.

Step 7: Fix the bad endpoint

Now correct the URL:

`https://jsonplaceholder.typicode.com/posts`

Click **Send** again.

Result:

Status: 200 OK


<https://jsonplaceholder.typicode.com/posts>
Save

GET
https://jsonplaceholder.typicode.com/posts
Send

Params
Authorization
Headers (6)
Body
Pre-request Script
Tests
Settings
Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body
Cookies
Headers (24)
Test Results
200 OK
67 ms
7.98 KB
Save Response

Pretty
Raw
Preview
Visualize
JSON

```

1  {
2    "userId": 1,
3    "id": 1,
4    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae
6  },
7  {
8    "userId": 1,
9    "id": 2,
10   "title": "qui est esse",
11   "body": "..."

```

Screenshot 7: Corrected endpoint returning a successful 200 OK response.

Step 8: Create a wrong-method example

GET vs POST.

Set method to:

GET

Use URL:

<https://jsonplaceholder.typicode.com/posts>

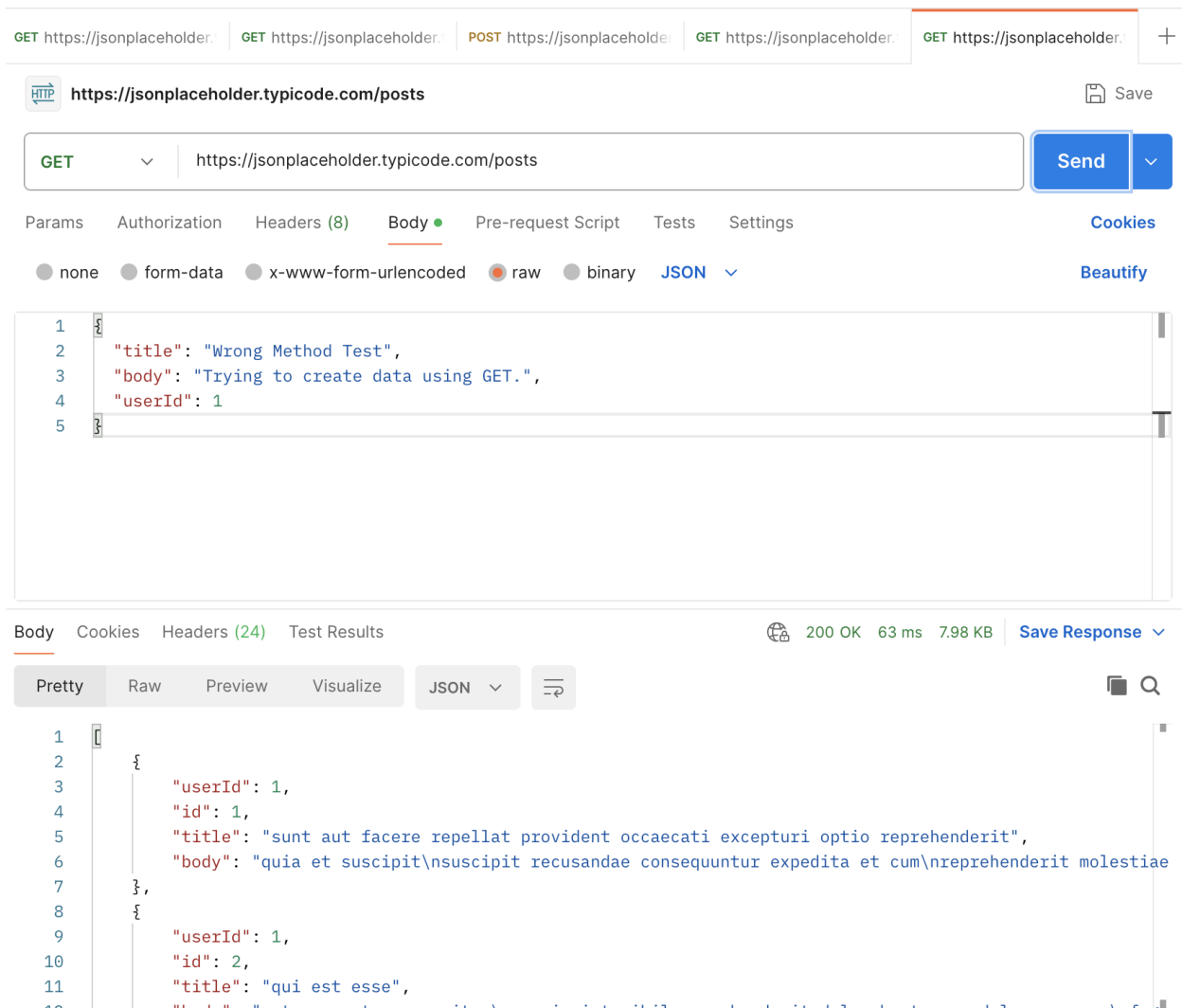
Go to **Body > raw > JSON** and paste:

```
{
  "title": "Wrong Method Test",
```

```
"body": "Trying to create data using GET.",
"userId": 1
}
```

Click **Send**.

GET is used to retrieve data. To create data, the correct method is POST.



Screenshot 8: Incorrect request method used when trying to create a new post.

Then change method to:

POST

Keeping the same URL and body.

Click **Send**.

Result:

Status: 201 Created

The screenshot displays the Postman interface for a POST request to the endpoint `https://jsonplaceholder.typicode.com/posts`. The request body is a JSON object: `{ "title": "Wrong Method Test", "body": "Trying to create data using GET.", "userId": 1 }`. The response status is `201 Created` with a response time of `120 ms` and a size of `1.3 KB`. The response body is a JSON object: `{ "title": "Wrong Method Test", "body": "Trying to create data using GET.", "userId": 1, "id": 101 }`.

HTTP `https://jsonplaceholder.typicode.com/posts` Save

POST `https://jsonplaceholder.typicode.com/posts` Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "title": "Wrong Method Test",
3   "body": "Trying to create data using GET.",
4   "userId": 1
5 }
```

Body Cookies Headers (25) Test Results 201 Created 120 ms 1.3 KB Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "title": "Wrong Method Test",
3   "body": "Trying to create data using GET.",
4   "userId": 1,
5   "id": 101
6 }
```

Screenshot 9: Corrected POST request successfully creating a test post.

7. Key Findings

- The `/posts` endpoint successfully returned JSON data using a GET request.
- A specific post could be retrieved using `/posts/1`.
- A POST request successfully created a test post and returned `201 Created`.
- A misspelled endpoint caused a failed request.
- Correcting the endpoint path resolved the issue.
- Using the correct HTTP method is important when troubleshooting API requests.
- Postman is useful for reproducing customer issues and validating API behavior.

8. Analysis

This lab demonstrated how a Technical Support Specialist can troubleshoot API-related customer issues in a structured way. I verified the API endpoint, confirmed the request method, reviewed the response status code, examined the JSON response body, and documented each troubleshooting step.

The most important part of the troubleshooting process was isolating whether the issue came from the endpoint, request method, request body, or response data. By testing both successful and failed requests, I practiced the same type of thinking needed in a SaaS technical support environment.

This lab also showed the importance of communicating technical findings clearly. A customer may not understand API terms like endpoint, JSON, or status code, so the support response must explain the issue in simple step-by-step language.

9. IT Support Relevance

This lab is relevant to IT support and technical support roles because many SaaS products use APIs to connect systems, transfer data, and support integrations. Technical Support Specialists may need to test API requests, reproduce customer issues, read error responses, document findings, and escalate issues with complete context.

This lab supports job responsibilities such as:

- Troubleshooting basic technical issues.
- Supporting software applications.
- Documenting customer interactions and resolutions.
- Explaining solutions clearly.
- Escalating technical issues with complete troubleshooting context.
- Building product and integration knowledge.

10. Skills Demonstrated

- API troubleshooting
- Postman
- REST API basics
- GET and POST requests
- JSON response review
- HTTP status code interpretation
- Endpoint validation
- Request method troubleshooting
- Technical documentation
- Ticket-style case notes
- Customer-focused communication

- Step-by-step support guidance
- Problem solving
- SaaS support fundamentals

11. Ticket-Style Summary

Ticket Title

Customer Unable to Retrieve API Data Due to Incorrect Endpoint

Ticket Type

Technical Support / API Troubleshooting

Priority

Medium

Customer Issue

Customer reported that their API request was not returning the expected data from the posts endpoint.

Environment

- Tool: Postman
- API: JSONPlaceholder
- Endpoint: `/posts`
- Method Tested: GET and POST
- Authentication: Not required

Steps to Reproduce

1. Open Postman.
2. Send GET request to:

`https://jsonplaceholder.typicode.com/postss`

3. Review response status.
4. Send corrected GET request to:

`https://jsonplaceholder.typicode.com/posts`

5. Confirm successful JSON response.

Troubleshooting Steps

1. Verified the API base URL.
2. Checked the endpoint path.
3. Compared incorrect endpoint `/postss` with correct endpoint `/posts`.
4. Sent a GET request to the corrected endpoint.
5. Confirmed successful `200 OK` response.
6. Reviewed JSON response fields.
7. Tested POST request with JSON body.
8. Confirmed successful `201 Created` response.

Root Cause

The API request failed because the endpoint path was typed incorrectly.

Resolution

Corrected the endpoint from:

`/postss`

to:

`/posts`

After correcting the endpoint, the request returned a successful JSON response.

Customer-Friendly Explanation

The issue was caused by a small typo in the API endpoint. The request was sent to `/postss`, but the correct endpoint is `/posts`. After updating the endpoint, the API returned the expected data successfully.

Status

Resolved

Follow-Up Recommendation

Use saved Postman requests or API documentation to reduce endpoint typing errors in future tests.

12. Lessons Learned

This lab helped me understand how APIs are tested and troubleshot in a technical support environment. I learned how to send GET and POST requests, read JSON responses, identify incorrect endpoints, and explain the resolution clearly. I also learned that small request errors, such as a misspelled endpoint or incorrect method, can cause customer-facing issues.

The biggest takeaway is that API troubleshooting requires a step-by-step process: confirm the URL, check the method, review the request body, read the status code, inspect the response, and document the resolution. This process is directly useful for technical support roles that involve SaaS products, integrations, and customer troubleshooting.